

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»

ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ И
КОМПЬЮТЕРНОЙ ТЕХНИКИ

ЛАБОРАТОРНАЯ РАБОТА №7
по дисциплине
«ОСНОВЫ ПРОФЕССИОНАЛЬНОЙ ДЕЯТЕЛЬНОСТИ»

Синтез команд БЭВМ

Вариант №36646

Выполнил:

Студент группы Р3107

Добрышкин Владимир Александрович

Проверил:

Осипов Святослав Владимирович

СОДЕРЖАНИЕ

1. Задание	3
2. Этапы выполнения	4
2.1. Синтезируемая программа	4
2.1.1. Код программы	4
2.1.2. Трассировка микропрограммы после выполнения команды	5
2.2. Текст тестирующей программы на языке Ассемблера БЭВМ	6
2.3. Разработанная и проверенная на БЭВМ методика проверки ...	11
2.3.1. Представленные тесты	11
2.3.2. Методика проверки программы	11
3. Вывод	12

ЗАДАНИЕ

Скриншот с se.ifmo

Лабораторная работа №7

Синтезировать цикл исполнения для выданных преподавателем команд. Разработать тестовые программы, которые проверяют каждую из синтезированных команд. Загрузить в микропрограммную память БЭВМ циклы исполнения синтезированных команд, загрузить в основную память БЭВМ тестовые программы. Проверить и отладить разработанные тестовые программы и микропрограммы.

Введите номер варианта

1. SHR - сдвиг аккумулятора вправо, 15 разряд заполняется значением 0. Признаки N/Z/V/C не устанавливать
2. Код операции - 0F01
3. Тестовая программа должна начинаться с адреса 034A₁₆

Текст задания

Синтезировать цикл исполнения для выданных преподавателем команд. Разработать тестовые программы, которые проверяют каждую из синтезированных команд. Загрузить в микропрограммную память БЭВМ циклы исполнения синтезированных команд, загрузить в основную память БЭВМ тестовые программы. Проверить и отладить разработанные тестовые программы и микропрограммы.

1. SHR - сдвиг аккумулятора вправо, 15 разряд заполняется значением 0. Признаки N/Z/V/C не устанавливать
2. Код операции – 0F01
3. Тестовая программа должна начинаться с адреса 034A₁₆

ЭТАПЫ ВЫПОЛНЕНИЯ

СИНТЕЗИРУЕМАЯ ПРОГРАММА

Код программы

```
ma BB
ma BB
mw 81F0014002
ma F0
ma F0
mw 0020209040
mw 0010180010
mw 0040009020
mw 80C4101040
```

Адрес ячейки	Новый код МК	Комментарий
BB	81E0014002	if CR(8) = 1 then GOTO F0
Цикл исполнения команды SHR		
F0	0020209040	PS $\rightarrow$ BR, C; Сохраняем регистр состояния в буферный регистр и обнуляем C
F1	0010180010	ROR(AC) $\rightarrow$ AC, C; Циклический сдвиг вправо для того, чтобы вернуть исходное число с обнуленным C
F2	0040009020	BR $\rightarrow$ PS; Возврат регистра состояния
F3	80C4101040	GOTO INT @ C4; Завершение цикла выполнения команды, переход к циклу прерываний

ТРАССИРОВКА МИКРОПРОГРАММЫ ПОСЛЕ ВЫПОЛНЕНИЯ КОМАНДЫ

Адр	МК	IP	CR	AR	DR	SP	BR	AC	NZVC	СчМК
F0	0020209040	013	0100	013	0100	000	0000	003F	0000	F1
F1	0010180010	013	0100	013	0100	000	0000	007F	0000	F2
F2	0040009020	013	0100	013	0100	000	0000	003F	0000	F3
F5	80C4101040	013	0100	013	0100	000	0000	001F	0000	C4

ТЕКСТ ТЕСТИРУЮЩЕЙ ПРОГРАММЫ НА ЯЗЫКЕ АССЕМБЛЕРА БЭВМ

```
org 0x34a
```

```
sta1:  word    0x0000    ; результат теста1 (0/1)
sta2:  word    0x0000    ; результат теста2 (0/1)
sta3:  word    0x0000    ; результат теста3 (0/1)
sta4:  word    0x0000    ; результат теста4 (0/1)
sta5:  word    0x0000    ; результат теста5 (0/1)
sta6:  word    0x0000    ; результат теста6 (0/1)
```

```
; Тест 1 -- подаётся нулевое значение
```

```
; out: 0, NZVC = const
```

```
inp1:  word    0x0000
```

```
ans1:  word    0x0000
```

```
; Тест 2 -- подаётся значение с AC_0 = 0 и AC_15 = 0
```

```
; out: сдвиг вправо, NZVC = const
```

```
inp2:  word    0x7bcc
```

```
ans2:  word    0x3de6
```

```
; Тест 3 -- подаётся значение с AC_0 = 0 и AC_15 = 1
```

```
; out: сдвиг вправо, NZVC = const
```

```
inp3:  word    0x8bcd
```

```
ans3:  word    0x45e6
```

```
; Тест 4 -- подаётся значение с AC_0 = 1 и AC_15 = 0
```

```
; out: сдвиг вправо, NZVC = const
```

```
inp4:  word    0x7bcd
```

```
ans4:  word    0x3de6
```

```
; Тест 5 -- подаётся значение с AC_0 = 1 и AC_15 = 1
```

```
; out: сдвиг вправо, NZVC = const
```

```
inp5:  word    0x8bcd
```

```
ans5:  word    0x45e6
```

```
; Тест 6 -- несколько последовательных сдвигов
```

```
; out: сдвиг вправо, NZVC = const
```

```
inp6:  word    0xffff
```

```
ans6:  word    0x0fff
```

```
; В тестах 1-6 так же проверяются все пограничные значения
```

```
; (0x0000, 0xffff, 0xxx1)
```

```
; Внутри каждого теста так же проверятся, что NZVC=const
```

```
; Тесты первого типа (с одним сдвигом)
```

```
test_type1:
```

```
    ld        &3
```

```

        word    0x0f01

        push

        pushf
        call    $clean_ps

        pop
        swap

        cmp     &2
        bne     pext1

        ld      &3
        cmp     &0
        bne     pext1

        ld      #0x01
        jump    ext1

pext1:  cla

ext1:   swap
        pop

        swap
        st      &3
        pop

        swap
        pop

        swap
        pop

        ret

```

; Тесты шестого типа (с четырьмя сдвигами)

```

test_type6:
        ld      &3

        word    0x0f01
        word    0x0f01
        word    0x0f01
        word    0x0f01

        push

        pushf
        call    $clean_ps

```

```

    pop
    swap

    cmp    &2
    bne    pext6

    ld     &3
    cmp    &0
    bne    pext6

    ld     #0x01
    jump   ext6

pext6:  cla

ext6:   swap
        pop

        swap
        st     &3
        pop

        swap
        pop

        swap
        pop

        ret

start:
    ; 1
    ld     inp1      ; входные данные на тест 1
    push

    pushf      ; nzvc перед тестом 1
    call     $clean_ps

    ld     ans1      ; ответ на тест 1
    push

    call     $test_type1
    st     stal
    nop              ; точка отладки

    ; 2
    ld     inp2      ; входные данные на тест 2
    push

```



```

pushf          ; nzvc перед тестом 2
call           $clean_ps

ld             ans2      ; ответ на тест 2
push

call           $test_type1
st             sta2
nop           ; точка отладки

; 3
ld             inp3      ; входные данные на тест 3
push

pushf          ; nzvc перед тестом 3
call           $clean_ps

ld             ans3      ; ответ на тест 3
push

call           $test_type1
st             sta3
nop           ; точка отладки

; 4
ld             inp4      ; входные данные на тест 4
push

pushf          ; nzvc перед тестом 4
call           $clean_ps

ld             ans4      ; ответ на тест 4
push

call           $test_type1
st             sta4
nop           ; точка отладки

; 5
ld             inp5      ; входные данные на тест 5
push

pushf          ; nzvc перед тестом 5
call           $clean_ps

ld             ans5      ; ответ на тест 5
push

call           $test_type1
st             sta5

```

```

nop                ; точка отладки

; 6
ld    inp6         ; входные данные на тест 6
push

pushf              ; nzvc перед тестом 6
call   $clean_ps

ld    ans6         ; ответ на тест 6
push

call   $test_type6
st     sta6
nop                ; точка отладки

; Проверка на то прошли ли все тесты
cla

add     sta1
add     sta2
add     sta3
add     sta4
add     sta5
add     sta6

cmp     #0x06
bne     ex

ld     #0x01
st     allin

ex:      hlt

allin:   word      0x0000 ; прошли ли все тесты (0/1)

; Функция для очистки последних 12 битов значения стека после pushf
clean_ps:
ld     &1
and    #0x0F
st     &1
ret

```

РАЗРАБОТАННАЯ И ПРОВЕРЕННАЯ НА БЭВМ

МЕТОДИКА ПРОВЕРКИ

ПРЕДСТАВЛЕННЫЕ ТЕСТЫ

1. Тест на сдвиг нулевого значения
2. Тест на сдвиг без переноса, когда $AC_0 = 0$ и $AC_{15} = 0$
3. Тест на сдвиг без переноса, когда $AC_0 = 0$ и $AC_{15} = 1$
4. Тест на сдвиг с переносом, когда $AC_0 = 1$ и $AC_{15} = 0$
5. Тест на сдвиг с переносом, когда $AC_0 = 1$ и $AC_{15} = 1$
6. Тест на несколько последовательных сдвигов

МЕТОДИКА ПРОВЕРКИ ПРОГРАММЫ

1. Открыть БЭВМ в формате dual “java -jar -Dmode=dual bcomp-ng.jar”
2. Ввести микрокоманды через консоль

```
ma BB
ma BB
mw 81F0014002
ma F0
ma F0
mw 0020009040
mw 0010220010
mw 0010380010
mw 0010380010
mw 0040009020
mw 80C4101040
```

3. Поставить hlt, где нужно.
4. Скомпилировать и запустить код на ассемблере
5. Удостовериться что после прогона всех тестов в аккумуляторе лежит 0x1 (0x00 - ошибка).

ВЫВОД

В ходе выполнения лабораторной работы я изучил микропрограммное устройство БЭВМ, научился ГРАМОТНО тестировать программы в БЭВМ, подробнее вник во внутреннее устройство БЭВМ.